

FRC Scouting App — Project Handoff Summary (v3)

A complete, self-contained record of the offline FRC scouting desktop application build: goals, source material, architecture, what has been implemented and verified in every stage through Stage 7, and exactly what a future session needs to know to continue with no access to any original chat history. This document supersedes the earlier Stage-6 handoff PDF (v2) — it contains everything that PDF did, brought up to date, plus one new stage: **Stage 7**, which root-caused and fixed two real bugs in the Match Lookup expected-score calculation (the alliance-combination step, not the earlier Adjusted Score Variance work from Stages 5–6). If this document and the code ever disagree, treat the code as ground truth and update this document.

1. Project Goal

Ian is building a fully offline Python desktop application that replicates the logic of a complex FRC (FIRST Robotics Competition) scouting Google Spreadsheet as a standalone tool. Requirements:

- 100% offline at runtime — no network calls, no external API, no cloud database.
- Runs on both Windows and Mac using only standard, freely-licensed components.
- Local storage only (SQLite file on disk).
- Four required UI tabs, in this order: Match Lookup, Team Lookup, Pick List, Rankings — matching the first four tabs of the source Google Sheet, whose content must appear on the matching tabs.
- The remaining ~28 sheets in the source spreadsheet contain the backend calculation logic and must be replicated exactly, not approximated casually — the user has explicitly and repeatedly stressed correctness over speed (“make no mistakes”, “it is imperative this code is also correctly implemented”).
- The scoring/forecasting logic implements a custom, non-Monte-Carlo skewed-distribution statistical method for forecasting robot performance and predicting match outcomes, documented in a PDF titled “Scouting Algorithm Overview” (a.k.a. `Scouting_Algorithm_Overview_4.pdf`).

2. Source Material

2.1 The Google Sheet

Original file: “Copy of Worlds Scouting Sheet” (Google Sheets ID `1RcHI_oUH3XHg3i4FAZgDR5eB0Wuq5HtgWGzZlrreO7c`). It contains 32 tabs and roughly 215,687 individual formula/value cells. The first four tabs share the exact names of the four required UI tabs; the other 28 tabs are backend calculation sheets.

2.2 How the formulas were extracted

Google Sheets formulas do not survive a plain .xlsx export reliably (Google-only functions like QUERY, ARRAYFORMULA, IMPORTRANGE, FILTER, XLOOKUP, VSTACK can be flattened or dropped). To get every formula as durable plain text, an Apps Script was written and run directly inside the spreadsheet (Extensions → Apps Script) that:

- Iterates every sheet and every used cell, recording Sheet name, Cell (A1 notation), Value (display value), and Formula (raw formula text, or blank for non-formula cells) into a new tab called `FORMULA_EXPORT`.

- Is resumable — because Apps Script has a ~6-minute execution limit, and this workbook has 215k+ cells, the script checkpoints its progress (current sheet index, row index, output row) in Script Properties every 500 rows and on a 5m20s time budget, so re-running the same function (exportAllFormulas) continues exactly where it left off. Helper functions resetExport() and checkExportStatus() were also provided.

The FORMULA_EXPORT tab was then downloaded as CSV:

Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv (215,687 data rows, columns: Sheet, Cell, Value, Formula). This CSV is the ground-truth source for every formula translated into Python in this project, and the ground-truth source used to find and verify every bug described in Sections 9, 10, and 11 (Stages 5, 6, and 7) below. Keep this CSV — it is required to verify or extend any calculation.

2.3 The algorithm PDF

“Scouting Algorithm Overview” (Scouting_Algorithm_Overview_4.pdf, 17 pages) describes the underlying statistical method in prose/derivation form: fitting a skew-normal-like distribution to each team’s historical scores WITHOUT Monte Carlo simulation, by (1) picking a transform (sqrt / log10 / reciprocal, or their power-inverses for left-skewed data) that best symmetrizes a team’s score history, (2) fitting a normal PDF over the transformed axis using the transformed data’s own mean/stdev, (3) scaling the resulting curve so its peak is 1, with a hard cutoff around the 90th–98th percentile and a broadening adjustment for weak teams, and (4) combining three teammates’ curves and comparing two alliances’ combined curves to estimate win probability. Equation OCR in a couple of spots (e.g. the sigma_adj approximation) was ambiguous (missing radicals/exponents); ground truth was resolved by cross-checking against the exported formulas (2.2) instead of trusting the OCR text alone.

2.4 A note on a missing sheet

Several formulas in the Team Lookup sheet (team name, weak-to-defense notes, climb-confidence flags) reference a sheet called “Super Scouting” that does not exist anywhere in the 32-sheet export. The user confirmed this sheet is not needed and instructed it be ignored — those specific Team Lookup fields are intentionally left unimplemented. Unchanged since the original build.

3. Delivery Approach: Staged, Not One-Shot

Given the size (32 sheets, 215k+ cells) and the presence of genuinely novel statistical logic (not boilerplate spreadsheet math), the build was explicitly staged rather than attempted in one pass, to avoid silently getting the core algorithm wrong:

- **Stage 1** — the raw data pipeline (10 sheets). Complete and delivered.
- **Stage 2** — the skew-normal curve fitting + defense/compatibility stats (4 more sheets). Complete and delivered.
- **Stage 3** — power rankings, alliance combination, match odds, and the real Pick List / Rankings logic (remaining sheets). Complete and delivered, with one explicitly flagged area needing the user’s validation (see §6.5).
- **Stage 4** — Data Import Redesign: replaced the original three-CSV granular input model with a streamlined two-CSV model built around the user’s real data format, fixed a zero-collapse bug in the Adjusted Score calculation, and validated the full pipeline end-to-end against the user’s actual competition data for the first time. Full detail in Section 13.
- **Stage 5** — Adjusted Score Variance Bug Investigation: three real bugs found and fixed in the import/calculation chain between Score Variance and Adjusted Score Variance (Match Schedule authority during import, an occurrence-indexed lookup quirk in the source sheet’s own formulas, and a

narrow evidence-based typo-correction feature). Verified against the source spreadsheet's own cached formula values. Full detail in Section 9.

- **Stage 6** — Experiment: at the user's explicit request, Adjusted Score Variance was removed from the calculation pipeline entirely and replaced everywhere with plain Score Variance. The user reviewed the results and decided to keep this change — it is current, active, production behavior. Full detail in Section 10.
- **Stage 7 (this update)** — Expected Score Bug Investigation: the user reported Match Lookup's expected score coming out massively below real match scores, as if the alliance combination were only picking up one of the three teammates' scores. Root-caused and fixed two distinct, real bugs in `calculations/alliance_generator.py` — a branch-ordering bug that silently zeroed a teammate's contribution right next to its own peak, and an alliance-level axis that was truncated to the same 1–500 range used for a single robot's score even though a 3-robot combined score routinely exceeds that. Both were confirmed as real by cross-checking `FORMULA_EXPORT`, and Bug 2 was confirmed to exist in the *original Google Sheet itself*, not just this port. Validated against all 110 real matches with a recorded score. Full detail in Section 11.

4. Current File Structure

Delivered as a project folder `frc_scouting_app/` (most recently shared zipped as `frc_scouting_app_8.zip` — the zip shared at the end of the Stage 7 session is the current, authoritative version; if a newer-numbered zip has been shared since, that one is authoritative instead). This is the full, current layout:

```
frc_scouting_app/
main.py entry point -- run this
database.py SQLite schema (local file, no server)
data_import.py CSV importers for the 2 active input tables
(manual_raw_data, match_schedule) -- see
Section 13 for the base format spec and
Section 9 for the Stage 5 import-time fixes
(Match Schedule authority, evidence-based
typo correction)
requirements.txt numpy, scipy, pandas
README.md setup + full data-import format reference +
the Stage 5 bug writeups + the Stage 6
experiment writeup + the Stage 7 expected-
score bug writeup (kept in sync with this
PDF's Sections 9, 10, and 11)
calculations/
team_roster.py "Team Numbers"
raw_processing.py ACTIVE: build_raw_data_from_manual()
LEGACY/unused: "BPS", "Unfiltered Raw
Data", "Raw Data" (team_bps_averages,
augment_unfiltered_rows, build_raw_data)
matrix_stats.py shared Min/Max/STDEV/AVG/RRP/Skew helper
balls_scored.py "Auto Balls Scored", "Teleop Balls Scored"
score_variance.py "Score Variance" (ACTIVE, used everywhere
downstream as of Stage 6), "Raw Data
Processing" (ACTIVE - still needed for
Defense Adjustment/Average Compatability/
RDP), Match Schedule "Adjusted" cols
(ACTIVE, feeds Raw Data Processing),
"Adjusted Score Variance"
(build_adjusted_score_variance() - STILL
DEFINED, but UNUSED by pipeline.py as of
```

Stage 6, see Section 10). Includes the Stage 4 raw-score fallback, the Stage 5 occurrence-indexed lookup fix, and the Stage 5 15-match cap fix. Untouched by Stage 7.

skew_methods.py "SQRT/Log10/1-over-X Method", "Skew Adjusted Spreads" -- as of Stage 6, fed by Score Variance, not Adjusted Score Variance (parameters renamed accordingly). Untouched by Stage 7.

transforms.py shared skew-transform math (Stage 1&2)

distributions.py "Skew Distributions", "Normalized Distributions" -- as of Stage 6, fed by Score Variance. Its X_AXIS_MAX (500) constant is still the single-robot axis only -- Stage 7 gave the alliance-level curve its OWN, separate, wider axis instead of changing this one.

defense.py "Defense Calculations", "Defense Adjustment", "Average Compatability"

power_rankings.py "ROP", "RDP", "RPP" -- ROP's build_rop() as of Stage 6 takes a `score\_variance` parameter (renamed from `adjusted\_score\_variance`)

team_ability.py "Team Ability Distribution"

pick_list.py "Alliance Builder Filtering"

alliance_generator.py "Alliance Base Generator", "Blue/Red Alliance Generator" -- CHANGED

IN STAGE 7. Two real bugs fixed:

- (1) _split_branches() now reverses the ascending branch's point order to match the sheet's actual SORT(...,FALSE()) trick;
- (2) the combined-alliance empirical curve now uses its own ALLIANCE_X_AXIS_MAX (= 3 * X_AXIS_MAX) instead of reusing the single-robot X_AXIS_MAX (500). See Section 11 for the full writeup. The Stage 3 cross-labeling question (Section 6.5) is STILL open and untouched by Stage 7.

match_odds.py "Match Odds" -- unchanged code, but its output is materially different now since it consumes alliance_generator.py's fixed output.

ui/

app.py main window, menu, notebook of 4 tabs --

- Data menu simplified to 2 imports in Stage 4; import summary dialog extended in Stage 5 to report schedule_conflicts and likely_typo_corrections. Untouched by Stage 7.

match_lookup_tab.py tab 1: Match Lookup (live win odds) -- as of Stage 6, its team panel reads self.data.score_variance, not self.data.adjusted_score_variance. Its expected-score/win% numbers are materially different as of Stage 7, but the tab's own code did not need to change -- it just calls build_alliance_distribution() + compute_match_odds() as before.

```
team_lookup_tab.py tab 2: Team Lookup -- as of Stage 6, the
separate "Adjusted Score Variance" panel
has been removed. Unaffected by Stage 7
(Stage 7 only touches the alliance-level
combination, not single-team curves).
pick_list_tab.py tab 3: Pick List (real ranking logic).
Unaffected by Stage 7.
rankings_tab.py tab 4: Rankings (real RPP/ROP/RDP).
Unaffected by Stage 7.
data/
scouting.db created automatically on first run
```

All 32 source sheets have a corresponding Python implementation. Every .py file's docstring documents which sheet(s) it recreates and the exact formula it is based on — treat those docstrings as the authoritative, line-by-line spec, and treat them as current: `calculations/alliance_generator.py`'s docstring was substantially rewritten in Stage 7 to document both bugs and the reasoning behind each fix in detail; it is the single best place to start if you need to touch alliance-combination logic again.

5. Runtime / Environment

- Language: Python 3.11+ (standard installer from python.org on both platforms).
- GUI: tkinter (ships with the standard Python installer — on Mac, must be the official python.org installer, not a Tk-less Homebrew build).
- Local data store: SQLite via the built-in `sqlite3` module — a single file at `data/scouting.db`.
- Math dependencies: `numpy`, `scipy` (for `scipy.stats.norm.pdf` in the curve-fit), `pandas` — all BSD-licensed, all offline once pip-installed.
- Zero network calls anywhere in the codebase — confirmed by design, no HTTP/socket libraries are imported.
- Setup: `pip install -r requirements.txt` then `python main.py`.

6. What Has Been Implemented (Stage-by-Stage Detail)

6.1 Local data model

There are two actively used input tables, `manual_raw_data` and `match_schedule`, populated by two CSV imports (see Section 13 for the full format spec and Section 9 for the Stage 5 import-time fixes). The original `raw_scouting` and `bps_samples` tables and their import paths still exist in the codebase but are unused legacy code — nothing writes to them anymore. Every other one of the 32 sheets is a pure formula sheet with no independent input, so it is recreated as a pure Python calculation each time the pipeline runs — there is no need to store intermediate results in the database.

6.2 Stage 1 — Raw Data Pipeline (10 sheets)

- **Team Numbers** → `calculations/team_roster.py` — team roster + per-team match counts.
- **BPS** → `calculations/raw_processing.py::team_bps_averages()` — average of each team's manual samples. [LEGACY, unused since Stage 4]
- **Unfiltered Raw Data** derived columns (S-AC) → `calculations/raw_processing.py::_augment_unfiltered_rows()`. [LEGACY, unused since Stage 4]

- **Raw Data** → `calculations/raw_processing.py::build_raw_data()`. [LEGACY, unused since Stage 4 — the ACTIVE equivalent is `build_raw_data_from_manual()`]
- **Auto Balls Scored / Teleop Balls Scored** → `calculations/balls_scored.py` — per-team chronological pivot (up to 15 matches) with `Min(2nd-smallest)/Max(2nd-largest)/STDEV/AVG/RRP`, using `AVG/(STDEV+1)` (note the +1, unique to these two sheets). Still active, unchanged.
- **Score Variance** → `calculations/score_variance.py::build_score_variance()` — same per-team chronological pivot pattern over Raw Data's Score column, plus Skew, explicitly capped to each team's first 15 chronological matches (a Stage 5 fix). Independently verified against the source spreadsheet's cached values at 624/625 (99.8%) exact match. This is the value used everywhere downstream as of Stage 6.
- **Raw Data Processing** → `calculations/score_variance.py::build_raw_data_processing()` — per match, per alliance slot: computes each alliance's "performance ratio" (sum of 3 teammates' raw scores / sum of their individual averages), and records BOTH that alliance's own ratio AND the OPPOSING alliance's ratio for that same match (the opposing ratio becomes a defense proxy used later). As of Stage 5, this replicates the source sheet's own occurrence-count-based COUNTIF/XLOOKUP indirection instead of a direct (team, match) lookup, including its quirks when a team has a scheduled match with no scouted row. Still needed as of Stage 6/7, since its `own_alliance_ratio/opponent_alliance_ratio` outputs feed Defense Adjustment, Average Compatability, and RDP.
- **Match Schedule "Adjusted" columns (N–S)** → also in `score_variance.py` — apportions each match's actual recorded alliance score to the 3 individual robots, proportional to their (occurrence-indexed) raw scouted score share. Unchanged in Stage 5/6/7 beyond the Stage 5 occurrence-indexing fix.
- **Adjusted Score Variance** → `build_adjusted_score_variance()` — same pivot pattern over the Adjusted values, plus the 4-way skew-method picker. Still fully implemented and callable, and still verified to be a faithful port of the sheet's own formulas (94.4% exact match once known, non-code data issues are excluded). As of Stage 6, it is simply not called by `pipeline.py` anymore.
- **SQRT Method / Log10 Method / 1/X Method** → `calculations/skew_methods.py` — apply one of 4 transforms to reduce skew (raw / sqrt / log10 / reciprocal if skew>0, else raw / square / cube / quartic if skew≤0), sharing scalar transform math with `calculations/transforms.py`. As of Stage 6, takes a `score_variance` parameter (was `adjusted_score_variance`).
- **Skew Adjusted Spreads** → `skew_methods.py::build_skew_adjusted_spreads()` — for each team, picks whichever of the 4 candidate skew values (raw, sqrt, log10, 1/x) is closest to zero, and that method's transformed data becomes the team's final skew-normalized spread — the direct input to Stage 2's curve fit.

6.2a Skew-method picker

(used repeatedly throughout the project, unchanged): for a team's data, compute the moment-based skew of the raw values; if it's negative (left-skewed) the four candidate transforms are {raw, x^2 , x^3 , x^4 }; if non-negative, {raw, $\sqrt{x}$, $\log_{10}(x)$, $1/x$ }. Whichever candidate's resulting skew has the smallest absolute value is selected (method 1=raw, 2= $\sqrt{x}$, 3= $\log_{10}/x^3$, 4= $1/x/x^4$). This exact rule reappears at the alliance level in Stage 3 (with weighted/frequency-based stats instead of raw-array stats).

6.3 Stage 2 — Curve Fitting + Defense/Compatibility (4 sheets)

- **Skew Distributions** → `calculations/distributions.py::build_skew_distributions()` — for each team and each whole-number score x from 1 to `X_AXIS_MAX` (500), evaluates `scipy.stats.norm.pdf` at `transform(x)` using that team's Skew Adjusted Spreads mean/stdev, with $y=0$ above the team's 95th-percentile raw score (hard cutoff). As of Stage 6, its first parameter is `score_variance` (was `adjusted_score_variance`).

- **Normalized Distributions** → `build_normalized_distributions()` — scales each team's raw curve so its peak is 1; teams whose Score Variance mean is at or below the league-wide 27th percentile get their curve's y-values replaced with $\sqrt{y}$ (a broadening adjustment for weak/uncertain teams). Produces each team's mode (x of peak y) as a byproduct. This mode value is what Stage 7's branch-splitting logic in `alliance_generator.py` keys off of.
- **Defense Calculations** → `calculations/defense.py::build_defense_calculations()` — fits a linear trend (SLOPE) through each team's first 12 chronological raw scores, computes residuals from that trend, and derives a trimmed-mean “defensive variance” (dropping the single worst residual), then min-max-normalizes it across all teams into an RRP.
- **Defense Adjustment** → `defense.py::build_defense_adjustment()` — reuses Raw Data Processing's per-match “opponent alliance ratio” list per team with its own Min/Max/skew stats.
- **Average Compatability** → `defense.py::build_average_compatability()` — reuses the “own alliance ratio” list per team, but the sheet intentionally excludes the very first chronological match from all of its stats — replicated exactly.

6.4 Stage 3 — Power Rankings, Pick List, Alliance Odds

- **ROP** → `calculations/power_rankings.py::build_rop()` — a weighted blend of Score Variance RRP and Average Compatability RRP (weights 2:0.5); Auto/Teleop Balls Scored RRP feed only the STDEV denominator, not the weighted average itself.
- **RDP** → `build_rdp()` — combines (a) how close a team's average opponent-suppression ratio is to the league-best team's, weight 2, with (b) that team's Defense Calculations RRP, weight 1.
- **RPP** → `build_rpp()` — weighted blend of ROP (2) and RDP (1), normalized to the league max; also computes a boolean “defense susceptible” flag and a Euclidean “distance from the ideal (1,1) point” in ROP-RDP space.
- **Team Ability Distribution** → `calculations/team_ability.py` — league-wide RPP benchmarks: overall average, top-4 average, top-8 average, an “elims window” average, and “Goal Ally RPP” = average of the top-8 and elims-window averages. This becomes the quality bar for the real Pick List.
- **Alliance Builder Filtering** → `calculations/pick_list.py::build_pick_list()` — the real Pick List logic. Teams at/above the Goal Ally RPP bar are ranked by Score Variance AVG/STDEV (a consistency ratio); teams below the bar form a fallback list ranked by ROP. Final list = top tier then fallback tier.
- **Alliance Base Generator + Blue/Red Alliance Generator** → `calculations/alliance_generator.py::build_alliance_distribution()` — combines three teammates' Normalized Distributions curves into one alliance-level combined score distribution WITHOUT full convolution, by splitting each teammate's curve into an ascending and descending branch (at its mode) and combining branches rank-by-rank across teammates. **Two real bugs in this step were found and fixed in Stage 7 — see Section 11.** Before Stage 7: the ascending branch's point order didn't match the sheet's actual ordering, and the combined-alliance curve reused the single-robot's 1–500 axis, both of which caused the expected score to come out far below reality. Both are now fixed.
- **Match Odds** → `calculations/match_odds.py::compute_match_odds()` — for each integer score x, sums $P(\text{alliance A} \geq x) \times P(\text{alliance B} \leq x)$ across all x (and the mirror image for B), then normalizes the two totals against each other into a final win probability; also reports each alliance's expected score (weighted mean of its empirical curve). The `match_odds.py` code itself did not change in Stage 7, but its real-world output improved substantially because it consumes `alliance_generator.py`'s now-corrected empirical curves.

6.5 Known Uncertainty — the Blue/Red cross-labeling question — STILL OPEN

The Alliance Generator → Match Odds chain was the single most intricate part of the entire 32-sheet workbook. Internally, the SOURCE SHEET cross-labels its outputs: the tab literally named “Blue Alliance Generator” is actually built from the RED alliance's three combined curves (framed as “the score Blue needs to clear to win”), and vice versa for “Red Alliance Generator” / Blue's curves. This cross-labeling could NOT be confirmed with full certainty from static formula analysis alone.

Decision made (unchanged since the original build, and unchanged by Stage 7): this Python port builds each alliance's combined distribution directly from its OWN three teammates (self-referential, unambiguous) rather than the sheet's cross-referenced labeling. This is mathematically equivalent for producing a sound win probability and is far easier to read/maintain, but it is NOT a byte-for-byte guarantee of matching the sheet's literal implementation if that cross-labeling turns out to matter.

Stage 7 did not touch this decision or resolve this uncertainty. Stage 7 was scoped narrowly to the branch-combination math within `build_alliance_distribution()` (how three teammates' curves get summed into one alliance curve) and the axis size that combined curve gets evaluated over — both of which apply identically regardless of which alliance's curve is being labeled “Blue” vs “Red” internally. This remains the single most valuable next validation step for whoever picks this project up: pick a real match from the original Google Sheet, compare its Match Odds win% there against this app's Match Lookup tab for the same match, and report back. If they disagree meaningfully in a way Stage 7's fixes don't explain, the cross-labeling in `alliance_generator.py`'s `build_alliance_distribution()` is the next place to revisit. See Section 12 for more.

6.6 Other Explicit Assumptions / Configurable Constants

- **X_AXIS_MAX = 500** (`calculations/distributions.py`) — the SINGLE-ROBOT score curve is built over whole numbers 1–500, matching the source sheet exactly. Your real season data shows individual-robot 95th-percentile cutoffs as high as ~589, so if Team Lookup curves ever look clipped for your highest-scoring teams, raise this constant. As of Stage 7, this no longer affects the alliance-level combined curve (see next bullet) — raising it only affects single-robot curves.
- **ALLIANCE_X_AXIS_MAX = 3 × X_AXIS_MAX = 1500** (`calculations/alliance_generator.py`, new in Stage 7) — the 3-ROBOT COMBINED score curve used by Match Lookup is built over whole numbers 1–1500. This was previously (incorrectly) reusing X_AXIS_MAX (500) directly, truncating most of an alliance's real combined-score probability mass — see Section 11 for the full writeup. If X_AXIS_MAX is ever raised (per the bullet above), this constant will automatically scale with it, since it's defined as a multiple of X_AXIS_MAX rather than a separate hardcoded number.
- **ELIM_ALLIANCE_TEAMS = 24** (`calculations/team_ability.py`) — the source sheet switches its “elims window” between 24 and 32 teams based on a live per-event cell ('Pick List'!B2) whose value wasn't available during this build. 24 (= 8 alliances × 3, the standard FRC playoff bracket) is the default. Adjust per-event if using a non-standard bracket size. Unchanged.
- Exact-match score lookup in `alliance_generator.py`'s empirical-curve step mirrors the sheet's VLOOKUP exact-match behavior — integer scores with no exact combined sample land on density 0 rather than being smoothed/interpolated between neighbors. Unchanged by Stage 7.

7. Current UI State (all 4 tabs are fully wired, no placeholders)

- **Match Lookup** — pick a match number; shows both alliances' three teams with Score Variance avg/RRP, the actual recorded score, and (built live, not cached) expected score + win probability from `calculations/alliance_generator.py` + `match_odds.py`. No code in this tab changed in Stage 7, but the expected-score and win-probability numbers it displays are now materially more realistic, since they come from the corrected `alliance_generator.py`.

- **Team Lookup** — pick a team; shows Auto/Teleop Balls Scored, Score Variance, Skew Adjusted Spread (with chosen method), fitted-distribution mode + 95th-percentile cutoff, Defense Calculations/Adjustment, Average Compatability, and RPP/ROP/RDP/defense-susceptible/distance. Unaffected by Stage 7.
- **Pick List** — real Alliance Builder Filtering output: Top Picks (above the Goal Ally RPP bar, sorted by quality ratio) then Other Picks (below the bar, sorted by ROP). Unaffected by Stage 7.
- **Rankings** — every team sorted by RPP descending, with ROP, RDP, Distance, and the Weak-to-Defense flag shown per row. Unaffected by Stage 7.
- **Data menu** — unchanged since Stage 4: “Import Match Schedule CSV...”, “Import Raw Data CSV (Team + Score Breakdown)...”, a separator, “Set My Team Number...”, “Recalculate Now.” Unaffected by Stage 7.

8. Testing Performed So Far

- After every stage, a small hand-built synthetic SQLite dataset was inserted directly via sqlite3, and calculations/pipeline.py::build_all() was run end-to-end in a headless Python process to confirm it executes without exceptions and produces sane-looking numbers.
- Stage 3 was checked for: RPP/ROP/RDP all landing in a sensible [0,1]-ish range; the Pick List correctly splitting into tiers; Match Odds probabilities for a sample match summing to 1.0 with plausible expected scores.
- Stage 4 validated against the user's REAL competition data for the first time: Match Schedule CSV (110 matches, all imported) and Raw Data CSV (654 data rows, 625 successfully imported, 29 correctly auto-skipped blank/placeholder rows, 0 unresolved-alliance rows at the time), full pipeline run cleanly against it (75 teams, 74-entry Pick List), and a directional Match Odds sanity check for one real match against its actual recorded score.
- Stage 5 root-caused the discrepancy the user reported between the app's Adjusted Score Variance and the live Google Sheet's own calculation by re-importing the user's real CSVs through the production import functions and cross-checking every intermediate value cell-by-cell against FORMULA_EXPORT's own cached values. Final measured results: Score Variance 624/625 (99.8%) exact match; Adjusted Score Variance 561/594 (94.4%) exact match once two confirmed non-code-bug causes are excluded.
- Stage 6 re-ran the full pipeline end-to-end against the same real data after removing Adjusted Score Variance from the calculation chain: 75 teams, 74-entry Pick List (unchanged), Goal Ally RPP recomputed to ≈ 0.780 (was ≈ 0.773 in the original Stage 4 baseline), and a live Match Odds computation for match 1 completed with no errors.
- **Stage 7 (new)** re-ran the full pipeline end-to-end against the same real data again (no changes to team count, Pick List size, or any upstream calculation — Stage 7 only touched alliance_generator.py, which is downstream of everything else): still 75 teams, 74-entry Pick List, confirming no regression. Then, for all 110 real matches with a recorded official score, computed each alliance's expected score via build_alliance_distribution() + compute_match_odds() both BEFORE and AFTER the Stage 7 fixes and compared against the real recorded scores: mean absolute error between expected and actual alliance score dropped from ≈ 175 points to ≈ 100 points, and win-direction accuracy (does the predicted favorite match the actual winner) rose from 67% (74/110) to 69% (76/110). A single worked example (match 1: red 4253/4099/6436 vs. blue 1796/5549/604, actual scores red 263 / blue 642) went from an expected R287/B335 (barely distinguishing the two alliances) to R412/B545 (correctly favoring blue by a wide margin) after the Stage 7 fixes.
- Every .py file continues to be syntax-checked (ast.parse) after every round of edits, including all Stage 5, 6, and 7 changes.

- Still not done: the Tkinter GUI has never been visually rendered/smoke-tested (the build sandbox has no display and python3-tk could not be installed there — no network access to fetch it). Logic was reviewed by hand instead. Flag anything that looks visually off once run on a real machine.
- Still not done: no validation against the ACTUAL Google Sheet's live Match Odds output for a real match (\$6.5). This remains the single highest-value next step — see Section 12.

9. Stage 5 — Adjusted Score Variance Bug Investigation & Fixes

Symptom reported by the user: Adjusted Score Variance, as calculated by the app, differed from what the live Google Sheet calculated for the same data. Score Variance (the step immediately before it) was confirmed correct by the user's own testing. This meant the bug had to be somewhere in the chain between Score Variance and Adjusted Score Variance: Raw Data Processing, the Match Schedule "Adjusted" columns, or Adjusted Score Variance's own pivot.

Method: the user's real CSVs (Untitled_spreadsheet_-_Raw_Data.csv, Untitled_spreadsheet_-_Match_Schedule.csv) were imported through the actual production import functions (data_import.import_match_schedule_csv, data_import.import_raw_data_csv) into a scratch SQLite database, and every intermediate value the pipeline produced was compared cell-by-cell against Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv's own cached Value column for the matching Sheet/Cell. Confirmed early on: the FORMULA_EXPORT CSV was generated from the exact same underlying dataset as the user's Raw Data/Match Schedule CSVs (matching team numbers and scores for match 1 across both sources), so a direct comparison is valid.

9.1 Bug 1 — Match Schedule not treated as authoritative during Raw Data import

Root cause: import_raw_data_csv() used to trust each row's own Alliance/Position cell before checking the already-imported match_schedule table, and would unconditionally upsert that (sometimes unreliable) value into match_schedule. In the user's real export, the Position column does not reliably reflect the true official Red1-3/Blue1-3 slot — it looks like scouts sometimes recorded it as "the Nth robot I personally scouted" rather than the official alliance slot. A concrete example found during testing: match 87's true roster (Red: 10935, 88, 3044 / Blue: 1540, 2783, 6436, per the authoritative Match Schedule CSV) was scrambled into (Red: 88, 6436, 2783 / Blue: 3354, 10935, 2783) purely because several Raw Data rows' own Position cells disagreed with the schedule.

Fix: import_raw_data_csv() now always tries a match_schedule lookup for a row's team+match first (via the existing helper _lookup_alliance_from_schedule()), and only falls back to that row's own Alliance/Position cell if the lookup fails. The match_schedule upsert is now only performed in that fallback case.

9.2 Bug 2 — Score Variance not capped to 15 matches

build_score_variance() did not cap each team's chronological score history to 15 matches, while the source sheet's Score Variance tab only has 15 match columns (B:P). This matters because Raw Data Processing's occurrence-indexed lookup (Bug 3 below) depends on this cap matching exactly. Fix: build_score_variance() now truncates each team's value list to its first 15 chronological matches.

9.3 Bug 3 — the occurrence-indexed lookup (a real quirk of the source sheet, now replicated)

The sheet's Raw Data Processing columns do NOT look up "this team's raw score for this match" by a direct (team, match) key. The actual formula is: COUNTIF({\$A\$3:\$C<row>,\$K\$3:\$M<row>}, <team>) → XLOOKUP(that count, Score Variance's header row 1–15, that team's Score Variance row). In words: "how many times has this team appeared anywhere in Match Schedule's 6 team columns, from the top of the schedule through this row?" gives an occurrence index K, and the sheet then takes the K-th value from that team's Score Variance history (chronological, capped at 15).

Confirmed as a real, reproducible behavior of the source sheet itself by tracing a specific example: team 687 is scheduled for 9 matches but has only 8 Raw Data rows (missing match 51); at match 99 (its 8th scheduled appearance), the sheet's own cached Raw Data Processing value for team 687 (70.19661809) turned out to

exactly equal team 687's real score from match 108 — its 8th actually scouted match. Fix: `build_raw_data_processing()` was rewritten to replicate this occurrence-indexed lookup exactly, including its handling of a team appearing twice within the same match row (both slots receive the same occurrence index, matching the sheet's COUNTIF range).

9.4 A fourth issue found via user feedback — evidence-based typo correction

After Bug 1's fix shipped, the user reported that Team Lookup now showed a “phantom” team, 325, that wasn't a real team. Investigation traced this to match 47: the Match Schedule CSV listed team 325 as Blue 3, while the Raw Data CSV had a row for team 3256 at that same match/slot. Team 325 appeared exactly once with zero corroborating raw data anywhere; team 3256 appeared in 8 other matches with consistent, real data. The user independently confirmed this was a typo (“325” being “3256” with the trailing digit dropped) that they will correct for future exports.

Fix implemented (and confirmed correct): `import_raw_data_csv()` now checks, for every conflict it detects, whether the team currently on record is a total “orphan” (appears exactly once, zero corroborating raw-data rows anywhere) while the conflicting row's team is well-established (at least 3 other rows for it elsewhere in the same CSV). Only in that narrow, lopsided case is the schedule slot corrected automatically; every more ambiguous conflict (34 found in the user's real data) is left unresolved-but-reported via `schedule_conflicts / schedule_conflict_examples`. Corrections are always reported via `likely_typo_corrections / likely_typo_correction_examples`, never silent.

An important caution surfaced during this work: matching a cached ground-truth snapshot and being factually correct are not always the same thing when the snapshot itself was generated from imperfect source data — FORMULA_EXPORT's own Match Schedule cell (G48) still shows the “325” typo, since it was captured before the correction. Always check the most authoritative source available (here, the user directly) before concluding an automated “fix” is wrong just because it diverges from a cache.

9.5 Validation results

With all of the above fixes in place, importing the user's real CSVs produces: 625 Raw Data rows imported, 29 correctly skipped as blank/placeholder rows, 0 unresolved-alliance rows, 34 `schedule_conflicts` (correctly left unresolved and reported), and 1 `likely_typo_corrections` entry (match 47: 325 → 3256, confirmed correct by the user). The full pipeline then produces exactly 75 teams and a 74-entry Pick List. Comparing every team's full chronological history against FORMULA_EXPORT's cached values: Score Variance 624/625 (99.8%) exact match; Adjusted Score Variance 564/622 (90.7%) exact match overall, rising to 561/594 (94.4%) once two confirmed non-code-bug causes are excluded (an isolated 3-row copy-paste error in the source spreadsheet's own formulas at matches 22–24, and sample Raw Data CSV completeness gaps confirmed present in FORMULA_EXPORT but missing from the sample CSV).

9.6 Code changes summary (Stage 5)

`data_import.py` — `import_raw_data_csv()`: Match Schedule lookup now tried before the row's own Alliance/Position cell; `match_schedule` upsert only performed on fallback; added the evidence-based typo-correction check; return dict extended with `schedule_conflicts`, `schedule_conflict_examples`, `likely_typo_corrections`, `likely_typo_correction_examples`. `calculations/score_variance.py` — `build_score_variance()` truncates to 15 matches; `build_raw_data_processing()` rewritten for the occurrence-indexed lookup. `ui/app.py` — post-import message box extended. `README.md` — new sections documenting all three fixes.

10. Stage 6 — Experiment: Adjusted Score Variance Removed

10.1 Rationale

At the user's explicit request. Real scouting data is human-entered and will always have holes in it — exactly the conditions Adjusted Score Variance's occurrence-indexed lookup (Section 9.3) is most sensitive to, since a single missing scouted match for a team silently shifts every later lookup for that team. Score Variance, by contrast, is simpler and was independently verified at 99.8% exact match. The user's own testing on the live Google Sheet found Adjusted Score Variance was only ever a minor adjustment on top of Score Variance.

Current status: the user has reviewed the results of this experiment and decided to keep the change. This is current, active, production behavior — not a proposal.

10.2 What changed

In `calculations/pipeline.py`: `build_all()` no longer calls `score_variance.build_adjusted_score_variance()` at all. Every downstream call that used to receive the Adjusted Score Variance dict now receives the plain Score Variance dict (`sv`) instead — `skew_methods.build_transform_methods(sv)`, `skew_methods.build_skew_adjusted_spreads(sv, ...)`, `distributions.build_skew_distributions(sv, sas)`, `distributions.build_normalized_distributions(sv, raw_curves)`, `power_rankings.build_rop(auto_bs, teleop_bs, sv, avg_compat)`. The `ComputedData` dataclass no longer has an `adjusted_score_variance` field at all (fails loudly rather than silently, by design). Parameter names were also renamed to `score_variance` in these functions for honesty/clarity.

In the UI: `ui/match_lookup_tab.py`'s per-team panel reads `self.data.score_variance`; its “Adj. Avg Score” column header was renamed to “Avg Score”. `ui/team_lookup_tab.py`'s separate “Adjusted Score Variance” panel was removed entirely.

10.3 What did NOT change

`calculations/score_variance.py::build_raw_data_processing()` still runs, unchanged — its own `_alliance_ratio` and `_opponent_alliance_ratio` outputs are what Defense Adjustment, Average Compatability, and RDP are built from, and none of that is Adjusted Score Variance. `build_adjusted_score_variance()` itself was not deleted — it's simply unused by `pipeline.py` now, and remains fully callable/testable in isolation.

`power_rankings.py::build_rpp()`'s variance field was already reading directly from plain Score Variance's STDEV before Stage 6 too.

10.4 How to revert

A small, localized change: in `calculations/pipeline.py`, add back `asv = score_variance.build_adjusted_score_variance(rdp_rows_raw)` and pass `asv` (instead of `sv`) into the five functions listed in 10.2; add `adjusted_score_variance` back to `ComputedData`; in `ui/team_lookup_tab.py` and `ui/match_lookup_tab.py`, point the removed/renamed panels back at `self.data.adjusted_score_variance`. This path is also documented in `README.md` and `pipeline.py`'s own module docstring.

10.5 A concrete before/after data point

Running the full pipeline against the user's real, Stage-5-corrected data (75 teams, `my_team_num=1796`): the Team Ability Distribution's Goal Ally RPP bar computed to approximately 0.836 immediately after the Stage 5 fixes (Adjusted Score Variance still active downstream), and approximately 0.780 after the Stage 6 change — compared to approximately 0.773 in the original Stage 4 baseline run. Presented as a reference point, not evidence either way about which approach is more correct. Team count (75) and Pick List size (74) were identical across all three runs, and remain identical after Stage 7 as well (Stage 7 is fully downstream of Team Ability Distribution / Pick List and does not affect them).

11. Stage 7 — Expected Score Bug Investigation & Fixes (this update)

11.1 Symptom reported

The user reported that Match Lookup's expected score was coming out massively below real match scores, and specifically noted it looked “as if the alliance combination were only picking up one of the three teammates' scores.” Everything else in the app (imports, Score Variance, Pick List, Rankings) appeared to be working correctly on the surface — the issue was isolated to the alliance-level expected-score/win-probability calculation shown on Match Lookup.

11.2 Investigation method

The investigation focused on `calculations/alliance_generator.py::build_alliance_distribution()`, the function that combines three teammates' Normalized Distributions curves into one alliance-level combined-score distribution. Rather than guessing, every step was cross-checked directly against FORMULA_EXPORT's “Alliance Base Generator”, “Blue Alliance Generator”, and “Match Odds” sheets — reading the actual formula text cell-by-cell, the same methodology used in Stage 5. Both bugs described below were confirmed as real quirks/limitations of the underlying spreadsheet mechanics, not assumptions.

11.3 Bug 1 — the ascending branch was combined in the wrong point order

Each teammate's normalized curve is unimodal: it rises to a peak of 1 at its mode, then falls. The sheet splits each teammate's curve at its own mode into two monotonic branches — an ascending branch (before the peak) and a descending branch (at/after the peak) — then combines all three teammates' branches rank-by-rank (the 1st ascending point of teammate A + the 1st ascending point of teammate B + the 1st ascending point of teammate C, and so on).

An earlier version of this port built the ascending branch in plain left-to-right (x-ascending) order: index 0 = the team's minimum score, the last index = the point just below its mode. Tracing the sheet's own formulas (“Alliance Base Generator”'s “Lower” columns, AE:AG) revealed this is backwards: the sheet collects the ascending-branch values in natural x order, then passes them through `SORT(values, SEQUENCE(n), FALSE())` — sorting by an increasing 1..n key in descending order, which simply REVERSES the list. So the sheet's real ascending (“Lower”) branch order starts right next to each team's own mode and moves outward to its minimum score — the mirror image of how the descending (“Upper”) branch is ordered (which the sheet leaves untouched, in natural order, mode outward — this part matched the port already).

With the branch order backwards, a teammate whose ascending branch was shorter than its teammates' (narrower spread on the low side) got zero-padded starting right next to its OWN mode — i.e. exactly its most heavily-weighted, most-likely-score region — once its shorter branch ran out. This silently dropped that teammate's real contribution from most of the alliance's higher-density combined samples, which is precisely the “only finding it for one out of three teams” symptom the user described. Fixed by reversing the ascending branch's point order in `_split_branches()` to match the sheet.

11.4 Bug 2 — the alliance-level combined curve was capped at the same 500-point axis as a single robot

Both this port and the ORIGINAL Google Sheet build the combined-alliance empirical curve (the “Blue/Red Alliance Generator” tabs' point axis, and this port's `empirical_curve` list) over the exact same 1–500 axis used for a single robot's score. But a 3-robot combined score can run up to 3x a single robot's max, and in this season's real data it very often does.

This was confirmed two independent ways, both directly from FORMULA_EXPORT: (1) “Alliance Base Generator”'s own cached combined-sample “Point Value” columns (W/AI, from a real, live example captured in the export) run as high as ~1079 with a median around ~670 — comfortably past 500 for the majority of samples; (2) the user's actual recorded Match Schedule scores (110 matches, 220 alliance-scores) range from 129 to 819 with a median of 439, and 78 of 220 (35%) exceed 500 outright. Since both the sheet's own “Blue Alliance Generator” E/F columns and this port's empirical_curve only ever look up integer scores in the range 1–500, any combined sample landing above 500 has nowhere to go and is silently dropped — not clipped to the boundary, just gone, along with its probability mass. This is a real limitation baked into the ORIGINAL spreadsheet's design (most likely because the “Blue/Red Alliance Generator” tabs were built by copying the single-robot curve's 1–500 row structure without extending it for a 3-robot sum), not something introduced by this port.

Fixed by giving the alliance-level curve its own, separate, wider axis: ALLIANCE_X_AXIS_MAX = 3 × X_AXIS_MAX (currently 1500), which is exactly the true maximum a 3-robot combination can reach given that each individual robot's x-values are themselves bounded to [X_AXIS_MIN, X_AXIS_MAX] by construction. This costs nothing extra — the additional rows are simply 0 wherever no real combination reaches that high — and it captures essentially all of the real combined-score probability mass instead of silently discarding most of it. Single-robot curves (feeding Team Lookup) are completely unaffected, since X_AXIS_MAX itself was left unchanged.

Bug 2 was the larger contributor of the two for any match whose real combined score already exceeded 500 — fixing Bug 1 alone barely moved those matches' expected scores, since most of the probability mass was still being cut off by the axis cap regardless of how correctly the samples were combined.

11.5 Validation results

Both fixes were validated together against all 110 real matches in the user's Match Schedule CSV that have a recorded official score, by computing each alliance's expected score via build_alliance_distribution() + compute_match_odds() and comparing against the real recorded red_score/blue_score:

- Mean absolute error between expected and actual alliance score: **≈175 points before → ≈100 points after.**
- Win-direction accuracy (does the alliance with the higher predicted win probability match the alliance that actually scored higher): **67% (74/110) before → 69% (76/110) after.**
- Team roster (75 teams) and Pick List size (74 entries) were identical before and after — expected, since Stage 7 only touches alliance_generator.py, which sits entirely downstream of the roster/pipeline/pick-list logic.
- Worked example, match 1 (red: 4253, 4099, 6436 vs. blue: 1796, 5549, 604; actual recorded scores red 263 / blue 642): expected scores went from R287/B335 (a nearly indistinguishable, low-magnitude guess that badly underestimated blue's real dominance) to R412/B545 (correctly and much more strongly favoring blue, in line with the lopsided actual result).

An average error around 100 points on real scores that range from under 200 to over 800 is a plausible result for a single-number forecast built from noisy, human-scouted data with real gaps in it (Section 9.5) — not perfect, but no longer “massively” off, and the predicted favorite now agrees with the actual winner roughly 7 times out of 10.

11.6 Code changes summary (Stage 7)

calculations/alliance_generator.py — _split_branches() now reverses the ascending branch's point order after collecting it (one added line, plus a substantially expanded docstring explaining why). A new module-level constant, ALLIANCE_X_AXIS_MAX = 3 * X_AXIS_MAX, was added and empirical_curve construction inside

`build_alliance_distribution()` now ranges over `X_AXIS_MIN..ALLIANCE_X_AXIS_MAX` instead of `X_AXIS_MIN..X_AXIS_MAX`. `AllianceDistribution`'s docstring/comment for the `empirical_curve` field was updated to reflect the new range. No other files needed changes — `match_odds.py`, the UI tabs, the database schema, and the import pipeline are all unaffected, since they either don't assume a fixed curve length or already just iterate whatever list `build_alliance_distribution()` hands them. `README.md` gained a new "Stage 7" section with this same writeup, and the note about `X_AXIS_MAX` in `README.md`'s configuration-constants section was updated to clarify that it now only governs single-robot curves.

11.7 An important caveat for anyone comparing against the live Google Sheet

Bug 2 (the axis truncation) is present in the **original Google Sheet itself**, confirmed directly from its own cached `FORMULA_EXPORT` values — not just in this port. This means the fix applied here is a deliberate, intentional divergence from literal sheet-quirk fidelity, made because the un-fixed behavior produced results that were clearly less accurate relative to real match outcomes, not because of any ambiguity in what the sheet's formulas do. If you ever compare this app's Match Lookup numbers directly against the live Google Sheet for the same match, expect the live sheet's own expected-score-style figures (if it exposes any) to still show the same underestimation this app used to show before Stage 7 — that is the sheet behaving as designed, not a sign this app is wrong. This is worth flagging to whoever maintains the original sheet if you're in a position to do so. Bug 1 (the branch order), by contrast, was purely a translation mistake in this port — the sheet's own formulas already implement the reversed order correctly, so this fix brings the port INTO agreement with the sheet rather than away from it.

12. Suggested Next Steps (in priority order)

- **Validate Match Odds against a real match from the original Google Sheet (§6.5)** — still the single highest-priority open item, unchanged since the original build and unresolved by Stage 7. Pick a match number, look up that match's Match Odds win% in the live Google Sheet, and compare it against this app's Match Lookup tab for the same match. This is also now the best available signal on whether the Blue/Red cross-labeling decision (§6.5) matters in practice, and a good real-world check on how well Stage 7's fixes generalize beyond the 110-match backtest in Section 11.5.
- Given the Stage 7 finding that individual-robot 95th-percentile cutoffs run as high as ~589 (Section 6.6), consider whether `X_AXIS_MAX` (currently 500, in `calculations/distributions.py`) should be raised for this season's data — this affects Team Lookup's single-robot curves specifically, is independent of the Stage 7 alliance-axis fix, and was flagged but not changed in this session since it wasn't the reported symptom.
- Keep an eye on real-world Pick List / Rankings quality over the next few events now that Stage 6 is live in production use, and compare informally against what the live Google Sheet's Pick List would have suggested for the same events, if practical. The revert path in Section 10.4 is there if this doesn't pan out.
- Spot-check a handful of other computed values (a team's RPP, a Pick List ordering) against the live Google Sheet using real historical scouting data now loaded in the app.
- Run the app's GUI end-to-end on an actual Windows or Mac machine with real data loaded, to confirm it renders correctly — this has still never been visually smoke-tested. Match Lookup's expected-score/win% numbers will look noticeably different (and more sensible) post-Stage-7; worth a specific glance.
- If the user encounters more Match Schedule typos like the match-47/team-325 case (Section 9.4), the existing `likely_typo_corrections` reporting should surface them automatically on import — but it's worth periodically reviewing the `schedule_conflicts` list too (34 currently-unresolved, genuinely-ambiguous conflicts in the user's existing data).
- Optional polish once validated: a results/history view comparing Match Odds predictions to actual outcomes over a season (Section 11.5's 110-match backtest logic would be a good starting point for this); plotting a team's fitted score curve or an alliance's combined distribution inline in the Team Lookup / Match Lookup tabs (the underlying data — `normalized_distributions`, `AllianceDistribution.empirical_curve` — already exists to plot, just needs a chart widget).
- If the “Super Scouting” sheet (§2.4) ever resurfaces, its formulas could be added to fill in Team Lookup's team-name / defense-notes / climb-confidence fields.
- If official alliance scores for additional matches become available, re-import an updated Match Schedule CSV — matches that already had a score keep using the exact apportioned formula, and any match that gains a score for the first time will automatically switch over from the raw-score fallback (Section 13.4) to the exact formula on the next Recalculate.

13. Data Import Redesign (Stage 4 detail, historical reference)

This section is preserved from the original Stage 4 handoff PDF as a historical/reference record of the base two-CSV import design. It describes the import model as it stood at the end of Stage 4, before the Stage 5 fixes in Section 9. Where Stage 5 changed the behavior described here (Match Schedule authority during conflicts, the evidence-based typo correction, the occurrence-indexed lookup replacing the direct (team, match) lookup mentioned below), Section 9 is the current, correct description — this section is kept for the parts that are still accurate (the CSV column formats, the two-row header detection, the recommended import order) and for context on why the two-CSV model looks the way it does. Nothing in Stage 6 or Stage 7 changed anything described in this section.

13.1 What was removed, and why

The original design had three CSV imports feeding three input tables: `raw_scouting` (granular, stopwatch-timed scouting entries), `bps_samples` (per-team ball-per-second calibration samples), and `match_schedule`. The user does not scout with a stopwatch — they pre-calculate each robot's full score breakdown themselves — so the `raw_scouting/bps_samples` path was never going to be used. At the user's explicit request, both were removed from the Data menu. Neither removal touched the underlying calculation code — `calculations/raw_processing.py` still contains `team_bps_averages()`, `_augment_unfiltered_rows()`, and `build_raw_data()` exactly as before, clearly marked in the module docstring as legacy/unused.

13.2 The active Raw Data CSV format

"Import Raw Data CSV (Team + Score Breakdown)..." is the sole source of Raw Data for the entire pipeline. One row per scouted robot per match. Implemented in `data_import.py::import_raw_data_csv()`. Recognized columns (header matching is case/space/underscore-insensitive):

- Team Number — required.
- Match Number — required.
- Alliance / Position — optional. Accepts 'R1'/'B2'/'red3'/'a bare 'Blue' (bare color with no slot digit is auto-numbered 1st/2nd/3rd in order of appearance for that match+color). As of Stage 5, this is only ever used as a fallback — the importer first checks whether Match Schedule already knows this team+match's slot, and only falls back to this cell if it doesn't.
- Auto Shooting / Auto Balls — auto balls scored.
- Auto Climb / Auto L1 — auto climb bonus points.
- Teleop Shooting / Teleop Balls — teleop balls scored.
- Teleop L1 / Teleop L2 / Teleop L3 — teleop climb level bonus.
- Score / Total Score — optional. If blank, it is derived from the breakdown columns, defaulting to 2 if that sum is exactly 0 — this exactly matches the original Raw Data sheet's own quirk.
- Notes — optional.

Header row detection (`data_import.py::_detect_raw_data_header()`) supports two formats automatically: a single-row header, or the two-row grouped header the user's actual export uses. Team numbers with thousands-separator commas (e.g. "1,796") are handled automatically. Blank/placeholder rows are silently skipped and counted, not treated as errors.

13.3 The active Match Schedule CSV format

"Import Match Schedule CSV..." is listed FIRST in the Data menu because it is the recommended import order. Implemented in `data_import.py::import_match_schedule_csv()`. 7 required columns (Match Number — header

may be blank; Red 1/2/3; Blue 1/2/3), 2 optional trailing columns (Red Score, Blue Score — if a CSV omits these entirely, existing values are left alone rather than nulled out). Uses a per-column partial upsert, so a scores-only re-import never wipes out team assignments already established, and vice versa.

13.4 Bug found and fixed in Stage 4: Adjusted Score zero-collapse

`build_raw_data_processing()` apportions each robot's raw scouted score against its alliance's OFFICIAL recorded match score. Originally, if that official score was still NULL, the apportionment formula silently coerced the missing score to 0.0, zeroing the Adjusted Score for every robot in that match. Fix (still in place, unchanged by Stage 5/6/7): for any match still missing an official `red_score/blue_score`, `build_raw_data_processing()` falls back to using each robot's raw scouted score AS the adjusted score directly (apportionment ratio of 1.0). The instant a real official score is imported for that match, the exact original apportioned formula automatically takes over on the next recalculation.

13.5 Final Data menu (unchanged through Stage 7)

```
Data
Import Match Schedule CSV...
Import Raw Data CSV (Team + Score Breakdown)...
-----
Set My Team Number...
Recalculate Now
```

13.6 Real-data validation performed in Stage 4

The user's actual Match Schedule CSV (110 matches) and Raw Data CSV (654 data rows) were imported through the production import functions for the first time: Match Schedule 110/110 rows imported successfully; Raw Data 625 rows imported, 29 correctly auto-skipped, 0 unresolved-alliance rows (at that time — Stage 5 later found and fixed real correctness issues in how those rows were being resolved). Full pipeline ran end-to-end with no errors: 75 teams, Goal Ally RPP ≈ 0.773 , 74-entry Pick List. Match Odds sanity check for match 1 produced a blue win probability of $\approx 61.4\%$ with expected scores of roughly 266 (red) / 300 (blue); the match's actual recorded final score was red 263 / blue 642 — the model's direction was correct even though the margin was badly underestimated, which is the exact symptom that led to the Stage 7 investigation in Section 11.

14. How to Resume This Project in a Fresh Session

Provide the following:

- This PDF (project context and every design decision made, through Stage 7).
- `Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv` — the ground-truth formula export, required for verifying any existing calculation or adding a new one. This is the same file used throughout Stage 5's and Stage 7's investigations; its `Sheet/Cell/Value/Formula` columns can be filtered directly to check any cell's real formula.
- The current `frc_scouting_app/` project folder (or its `.zip` — `frc_scouting_app_8.zip` as of the end of the Stage 7 session, or a newer-numbered zip if one has been shared since). Every `.py` file's docstring is the authoritative spec for what it implements; `alliance_generator.py`'s docstring in particular now documents both Stage 7 bugs and the reasoning behind each fix in detail.
- The user's real `Untitled_spreadsheet_-_Raw_Data.csv` and `Untitled_spreadsheet_-_Match_Schedule.csv` files (or newer versions of them) — these are the exact files used for every validation number in Sections 9, 10.5, and 11.5.

With those, a fresh session can:

- Re-run the real-data pipeline test to confirm nothing has regressed: re-import the two CSVs into a fresh `data/scouting.db` via `data_import.import_match_schedule_csv()` then `data_import.import_raw_data_csv()`, then call `calculations.pipeline.build_all(my_team_num=1796)`. Expect: 625 imported, 29 skipped_missing_ids, 0 skipped_ambiguous_alliance, 34 schedule_conflicts, 1 likely_typo_corrections (match 47, 325→3256), and a resulting 75-team roster with a 74-entry Pick List.
- Look up any cell's real formula by filtering the `FORMULA_EXPORT` CSV on the `Sheet` column, exactly as was done throughout Sections 9 and 11's investigations.
- Re-validate the Stage 7 fix specifically: for each of the 110 matches in `match_schedule` with a non-null `red_score/blue_score`, build each alliance's distribution via `calculations.alliance_generator.build_alliance_distribution()` (using the team roster from `match_schedule` and each team's `calculations.pipeline.build_all(...).normalized_distributions` entry), run `calculations.match_odds.compute_match_odds()` on the pair, and compare `expected_score_a/expected_score_b` against the real `red_score/blue_score`. Expect a mean absolute error around 100 points and a win-direction accuracy around 69% (76/110), matching Section 11.5.
- If asked to re-validate Adjusted Score Variance specifically (e.g. as part of deciding whether to revert Stage 6 per Section 10.4): call `calculations.score_variance.build_adjusted_score_variance()` directly (it still exists and works, it's just not wired into `pipeline.py` anymore) and compare against `FORMULA_EXPORT`'s "Adjusted Score Variance" sheet exactly as Section 9.5 describes.
- Cross-reference this PDF's Section 6 (calculation logic), Section 9 (Stage 5 bug fixes), Section 10 (Stage 6 experiment), Section 11 (Stage 7 expected-score bug fixes), and Section 13 (Stage 4 base import detail) to see exactly which Python function corresponds to which sheet or which CSV, which judgment calls were already made, and why — without needing to re-derive any of the analysis from scratch.

The single most valuable next action for whoever picks this up is still the outstanding Section 6.5 / Section 12 validation: compare this app's Match Odds output for a real match against that same match's Match Odds in the live Google Sheet. This is now also the best available check on whether Stage 7's fixes hold up beyond the historical 110-match backtest, and on whether the still-open Blue/Red cross-labeling question (Section 6.5) matters in practice. A secondary priority is keeping an eye on how the Stage 6 experiment (Section 10) performs in real-world use, since the user has decided to keep it as production behavior for now, and considering whether `X_AXIS_MAX` (Section 6.6) needs raising for this season's individual-robot score range.

End of summary. Generated to allow this project to be resumed with no access to the original chat history. This is the v3 revision, updated at the end of the Stage 7 (expected-score bug investigation and fixes) session to remain accurate and complete through Stage 7.